

1 Roteiro Prático 04 — Arquitetura Frontend Modular: ES6 Modules & Padrão em Camadas

Objetivos

Objetivos da Atividade (Módulo 2 — ES6 Modules & Padrão em Camadas)

Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Superar a arquitetura monolítica de arquivo único (`app.js`), organizando o frontend em múltiplos módulos especializados utilizando **ES6 Modules** (`import` e `export`);
- Declarar o ponto de entrada modular na página web através da diretiva:
`<script type="module" src="js/main.js"></script>`;
- Compreender e aplicar a separação estrita de responsabilidades no cliente em três camadas:
 - **Camada de Serviços (`services/`):** Regras de negócio, cálculos e persistência pura no `localStorage` sem acoplamento ao DOM;
 - **Camada de Visão (`views/`):** Renderização de cards, tabelas, modais e Toasts via *template literals* do Bootstrap 5;
 - **Camada Utilitária (`utils/`):** Funções puras reutilizáveis de formatação (moeda brasileira, telefone, máscaras);
 - **Orquestrador Central (`main.js`):** Captura de eventos do usuário e coordenação entre camadas.
- Implementar o **CRUD Completo** (*Create, Read, Update, Delete*) com persistência em armazenamento local;
- Implementar **validação visual no cliente** com feedback imediato ao usuário utilizando as classes nativas do Bootstrap 5 (`.is-valid`, `.is-invalid` e `.invalid-feedback`);
- Disparar notificações flutuantes dinâmicas (**Bootstrap Toasts**) para confirmar operações de cadastro, alteração e exclusão;
- Implementar filtragem reativa em tempo real por categoria de serviço na vitrine comunitária.

1.1 Introdução e Contextualização do Projeto

Nos Roteiros 1, 2 e 3, trabalhamos com scripts JavaScript concentrados em um único arquivo (`app.js`). Embora essa abordagem seja excelente para aprender os fundamentos, ela se torna inviável em aplicações reais de médio e grande porte, onde a mistura de seletores do DOM, regras de validação e

persistência gera o chamado *"código espaguete"*.

No **Roteiro Prático 04**, damos o passo definitivo rumo à arquitetura profissional de software no frontend. Desenvolveremos a **Vitrine Comunitária de Empreendedores Locais de Garopaba**, uma aplicação que permite cadastrar, listar, filtrar, editar e gerenciar serviços da comunidade local.

Este projeto serve também como a **base tecnológica inicial para a Ação Curricularizada de Extensão na SNCT (Semana Nacional de Ciência e Tecnologia)**.

1.2 Estrutura Arquitetural de Pastas

O projeto deve ser organizado na pasta `roteiro-4/exercicio-roteiro4/` seguindo rigorosamente a estrutura modular:

Árvore de Diretórios Modular do Projeto:

```
roteiro-4/exercicio-roteiro4/
|-- index.html      <-- Vitrine e forms (BS5)
|-- styles.css      <-- Estilos e transições
`-- js/
    |-- main.js     <-- Orquestrador central
    |-- services/
    |   `-- vitrineService.js <-- CRUD Storage
    |-- views/
    |   `-- vitrineView.js   <-- HTML dinâmico
    `-- utils/
        `-- formatters.js    <-- Formatações
```

1.3 Fundamentação Teórica: Como Funcionam os ES6 Modules

1.3.1 1. Declaração no HTML (type="module")

Ao declarar `<script type="module" src="js/main.js"></script>`, o navegador aplica comportamentos modernos essenciais:

- **Modo Estrito Automático ("use strict"):** Impede o uso de variáveis não declaradas e falhas silenciosas;
- **Escopo Fechado de Módulo:** Variáveis e funções criadas dentro de um arquivo não poluem o escopo global (window);
- **Carregamento Assíncrono com Adiamento (defer):** O script aguarda o parsing completo do DOM antes de executar.

1.3.2 2. Exportação e Importação (export e import)

Para compartilhar recursos entre arquivos, usamos a sintaxe explícita de módulos:

Exemplo de Exportação Nomeada e Importação:

```
// No arquivo js/utils/formatters.js
export function formatarTelefone(telefone) {
  return telefone.replace(
    /~(\d{2})(\d{5})(\d{4})/,
    '($1) $2-$3'
  );
}

// No arquivo js/main.js
import { formatarTelefone }
  from './utils/formatters.js';

// Resultado: (48) 99999-8888
console.log(formatarTelefone('48999998888'));
```

1.4 Passo a Passo Guiado do Roteiro

Passo 1: Estruturação dos Utilitários Puros (js/utils/formatters.js)

Crie o arquivo de funções utilitárias que realizam operações puras de transformação de dados:

```
// js/utils/formatters.js
export function formatarMoeda(valor) {
  const num = parseFloat(valor) || 0;
  return num.toLocaleString('pt-BR', {
    style: 'currency',
    currency: 'BRL'
  });
}

export function formatarTelefone(fone) {
  const digits = (fone || '').replace(/~\D/g, '');
  if (digits.length === 11) {
    return digits.replace(
      /~(\d{2})(\d{5})(\d{4})/,
      '($1) $2-$3'
    );
  }
  return fone;
}

export function escapeHtml(texto) {
  const div = document.createElement('div');
  div.textContent = texto;
  return div.innerHTML;
}
```

Passo 2: Camada de Dados e Serviços (js/services/vitrineService.js)

Esta camada gerencia o ciclo de vida dos dados no localStorage sem qualquer manipulação de elementos HTML (operações CRUD puras):

```
// js/services/vitrineService.js
const STORAGE_KEY = 'garopaba_vitrine_servicos';

export function obterServicos() {
  const dados = localStorage.getItem(STORAGE_KEY);
  return dados ? JSON.parse(dados) : [];
}

export function salvarServico(novo) {
  const servicos = obterServicos();
  const completo = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novo
  };
  servicos.unshift(completo);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return completo;
}

export function atualizarServico(id, dados) {
  const servicos = obterServicos();
  const idx = servicos.findIndex(s => s.id === id);
  if (idx !== -1) {
    servicos[idx] = { ...servicos[idx], ...dados,
      dataEdicao: new Date().toISOString() };
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return servicos[idx];
  }
  return null;
}

export function removerServico(id) {
  const servicos = obterServicos().filter(s => s.id !== id);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicos;
}
```

Passo 3.1: Template do Card de Serviço (js/views/vitrineView.js)

Gera a estrutura HTML de cada card individual com dados formatados e botões de ação:

```
// js/views/vitrineView.js (Parte 1: Template do Card)
import { formatarMoeda, formatarTelefone, escapeHtml }
  from '../utils/formatters.js';

export function criarCardHtml(s) {
  const tel = (s.telefone || '').replace(/\D/g, '');
  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card">
        <div class="card-body">
          <div class="d-flex justify-content-between mb-2">
            <span class="badge bg-success">
              ${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-primary">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>
          <h5 class="card-title fw-bold mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>
            ${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small">
            ${escapeHtml(s.descricao)}
          </p>
        </div>
        <div class="card-footer bg-transparent border-0 pt-0 pb-3
          d-flex justify-content-between align-items-center">
          <a href="https://wa.me/55${tel}" target="_blank"
            class="btn btn-sm btn-outline-success">
            <i class="bi bi-whatsapp me-1"></i>
            ${formatarTelefone(s.telefone)}
          </a>
          <div class="d-flex gap-1">
            <button class="btn btn-sm btn-outline-primary btn-editar"
              data-id="${s.id}" title="Editar">
              <i class="bi bi-pencil-square"></i>
            </button>
            <button class="btn btn-sm btn-outline-danger btn-excluir"
              data-id="${s.id}" title="Excluir">
              <i class="bi bi-trash"></i>
            </button>
          </div>
        </div>
      </div>
    </div>`;
}
```

Passo 3.2: Renderização da Vitrine e Estado Vazio

Itera sobre a coleção de serviços chamando a função de template e tratando o estado vazio:

```
// js/views/vitrineView.js (Parte 2: Renderização)
export function renderizarCards(servicos, container) {
  if (!servicos || servicos.length === 0) {
    container.innerHTML = `
      <div class="col-12 text-center py-5">
        <i class="bi bi-inbox fs-1 text-muted"></i>
        <h5 class="text-muted mt-2">
          Nenhum serviço cadastrado nesta categoria.
        </h5>
      </div>`;
    return;
  }
  container.innerHTML = servicos.map(criarCardHtml).join('');
}
```

Passo 3.3: Notificações Flutuantes (Bootstrap Toasts)

Dispara notificações assíncronas flutuantes através da API nativa do Bootstrap Toast:

```
// js/views/vitrineView.js (Parte 3: Toasts)
export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastBody = document.getElementById('toastMensagem');
  if (toastEl && toastBody) {
    toastBody.innerText = mensagem;
    toastEl.className =
      `toast align-items-center text-bg-${tipo} border-0`;
    const toast = bootstrap.Toast.getOrCreateInstance(toastEl);
    toast.show();
  }
}
```

Passo 4.1: Inicialização e Reatividade (js/main.js)

Configura as referências aos elementos do DOM e implementa a renderização reativa:

```
// js/main.js (Parte 1: Inicialização e Leitura)
import {
  obterServicos, salvarServico,
  atualizarServico, removerServico
} from './services/vitrineService.js';
import {
  renderizarCards, exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const modalEl = document.getElementById('modalCadastro');
  const servicoIdInput = document.getElementById('servicoId');

  function atualizarInterface() {
    const todos = obterServicos();
    const filtro = filtroCategoria.value;
    const filtrados = filtro === 'todas'
      ? todos
      : todos.filter(s => s.categoria === filtro);
    renderizarCards(filtrados, container);
    if (contadorEl) {
      contadorEl.innerText = `${todos.length} cadastrados`;
    }
  }

  filtroCategoria.addEventListener('change', atualizarInterface);
  atualizarInterface();
});
```

Passo 4.2: Validação e Submissão — Create & Update (js/main.js)

Gerencia a validação nativa do formulário e alterna entre novo cadastro e atualização:

```
// js/main.js (Parte 2: Submissão do Formulário)
form.addEventListener('submit', (e) => {
  e.preventDefault();
  if (!form.checkValidity()) {
    e.stopPropagation();
    form.classList.add('was-validated');
    return;
  }

  const idAtual = servicoIdInput ? servicoIdInput.value : '';
  const getVal = (id) => document.getElementById(id).value.trim();

  const dados = {
    nome: getVal('nome'),
    categoria: document.getElementById('categoria').value,
    bairro: getVal('bairro'),
    precoBase: parseFloat(getVal('precoBase')) || 0,
    telefone: getVal('telefone'),
    descricao: getVal('descricao')
  };

  if (idAtual) {
    atualizarServico(idAtual, dados);
    exibirToast('Serviço atualizado com sucesso!');
  } else {
    salvarServico(dados);
    exibirToast('Empreendimento cadastrado com sucesso!');
  }

  form.reset();
  if (servicoIdInput) servicoIdInput.value = '';
  form.classList.remove('was-validated');

  if (modalEl) {
    const modal = bootstrap.Modal.getInstance(modalEl);
    if (modal) modal.hide();
  }
  atualizarInterface();
});
```


Passo 4.3: Delegação de Eventos — Edit & Delete (js/main.js)

Escuta cliques nos botões dos cards para carregar dados no modal ou remover itens:

```
// js/main.js (Parte 3: Delegação de Cliques nos Cards)
container.addEventListener('click', (e) => {
  // 1. Operação Update (Edição)
  const btnEditar = e.target.closest('.btn-editar');
  if (btnEditar) {
    const id = btnEditar.getAttribute('data-id');
    const item = obterServicos().find(s => s.id === id);
    if (item) {
      if (servicoIdInput) servicoIdInput.value = item.id;
      document.getElementById('nome').value = item.nome;
      document.getElementById('categoria').value = item.categoria;
      document.getElementById('bairro').value = item.bairro;
      document.getElementById('precoBase').value = item.precoBase;
      document.getElementById('telefone').value = item.telefone;
      document.getElementById('descricao').value = item.descricao;

      if (modalEl) {
        const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
        modal.show();
      }
    }
    return;
  }

  // 2. Operação Delete (Exclusão)
  const btnExcluir = e.target.closest('.btn-excluir');
  if (btnExcluir) {
    const id = btnExcluir.getAttribute('data-id');
    if (confirm('Deseja realmente remover este serviço da vitrine?')) {
      removerServico(id);
      atualizarInterface();
      exibirToast('Serviço removido da vitrine.', 'warning');
    }
  }
});
}); // Fim do DOMContentLoaded
```

1.5 Checklist de Conclusão e Critérios Avaliativos

Antes de realizar o commit final no repositório, certifique-se de validar todos os itens:

- ☐ O projeto utiliza estritamente o padrão ES6 Modules com a tag:
`<script type="module" src="js/main.js"></script>`;
- ☐ A estrutura em pastas (js/services/, js/views/, js/utils/) foi rigorosamente respeitada;
- ☐ O arquivo vitrineService.js opera puramente sobre o localStorage sem manipular elementos do DOM;
- ☐ As quatro operações do CRUD (**Create, Read, Update e Delete**) foram implementadas e testadas com persistência no navegador;
- ☐ O formulário exibe validações visuais nativas do Bootstrap 5 (.is-invalid) quando campos obrigatórios não são preenchidos;
- ☐ A vitrine reage instantaneamente aos filtros de categoria sem recarregar a página;
- ☐ O código está devidamente publicado e em pleno funcionamento no **GitHub Pages**.